

Writing something as simple as a Hello World program can be quite a challenge in assembly. The above code cheats slightly by simply calling the C printf function for the bit that actually prints to screen. The rest of the code simply prepares to call printf and pass on the message 'hello, world'.

The 'section.data' segment of code we see above instructs the assembler to store 'hello, world' in memory and put its location in 'fmtStr'. After that the sub (subtract) statement, the lea (load effective address) statement, and the mov (move) statements put data and pointers in the right place so the printf function knows where to find them.

This is by no means the only assembly version of Hello World, not even for NASM.

```

• Sort
• isort:
•   %define a [ebp + 8]
•   %define n [ebp + 12]
•   enter 0, 0
•   pusha
•   mov ecx, 1
•   for:
•       mov ebx, ecx
•       imul ebx, 4
•       add ebx, a
•       mov ebx, [ebx]
•       mov edx, ecx
•       dec edx
•       while:
•           cmp edx, 0
•           jl while_quit
•           mov eax, edx
•           imul eax, 4
•           add eax, a
•           cmp ebx, [eax]
•           jge while_quit
•           mov esi, [eax]
•           mov dword [eax + 4], esi
•           dec edx
•           jmp while
•       while_quit:
•           mov [eax], ebx

```